

11.6.3 Titkosítás

Adott egy k bites üzenet $m \in \mathbb{F}_2^k$:

1. Generáljunk egy véletlenszerű hibavektort $e \in \mathbb{F}_2^n$ pontosan t darab 1-es bittel (Hamming-súly $\text{wt}(e) = t$).
2. A ciphertext:

$$c = m \cdot G' + e \in \mathbb{F}_2^n$$

11.6.4 Visszafejtés

A titkos kulcs birtokában:

1. Számoljuk ki: $c' = c \cdot P^{-1} = m \cdot S \cdot G + e \cdot P^{-1}$.
2. Mivel P permutáció, $e' = e \cdot P^{-1}$ szintén t súlyú. Továbbá $m \cdot S \cdot G$ a $\mathcal{C}$ kód egy kódszava.
3. A Goppa-kód hatékony dekódolójával (pl. Patterson-algoritmus) javítsuk a t hibát c' -ben, megkapva $m' = m \cdot S$.
4. Számoljuk ki az eredeti üzenetet: $m = m' \cdot S^{-1}$.

11.6.5 Biztonság

A támadónak a publikus kulcs G' ismeretében kellene dekódolnia – de G' egy véletlenszerűnek tűnő lineáris kód generátormátrixa, amelyhez nem ismert hatékony dekódoló. A biztonság két nehéz problémán alapul:

- **Általános dekódolási probléma:** Adott egy véletlenszerű $[n, k]$ lineáris kód és egy t hibával terhelt kódszó, a dekódolás NP-nehéz.
- **Kód-megkülönböztethetlenség:** A Goppa-kód álcázott generátormátrixa ($G' = S \cdot G \cdot P$) megkülönböztethetetlen egy véletlenszerű mátrixtól.

A legjobb ismert támadás az Information Set Decoding (ISD), amelynek futásideje exponenciális n -ben. Shor algoritmusa nem alkalmazható erre a problémára, Grover algoritmusa pedig csak négyzetgyök-gyorsítást ad, amit nagyobb paraméterek kompenzálnak.

A McEliece fő hátránya a publikus kulcs mérete: a Classic McEliece szabványjavaslat 128 bites posztkvantum biztonsághoz $n = 3488$, $k = 2720$, $t = 64$ paramétereket használ, ami ≈ 261 KB méretű publikus kulcsot eredményez [96]. Ez nagyságrendekkel nagyobb mint az ML-KEM néhány kilobájtos kulcsa, de cserébe a rendszer biztonsági garanciája a legerősebb az ismert posztkvantum sémák között.

A blokklánc ökoszisztéma szempontjából a kvantumfenyegetés különösen súlyos, mert a legtöbb hálózat – Bitcoin, Ethereum, Solana, Tezos – ECDSA vagy EdDSA aláírásokat használ a tranzakciók hitelesítésére. Ha egy támadó kvantumszámítógéppel ki tudja számolni a privát kulcsot a publikus kulcsból, akkor tetszőleges felhasználó nevében tud tranzakciót aláírni. Több blokklánc projekt már megkezdte a posztkvantum migrációt: az Ethereum kutatóközössége a kvantumrezisztens aláírások integrálásán dolgozik, és a NIST szabványok véglegesítése után várhatóan a többi hálózat is lépni fog. Ahogyan Babbush és társszerzői fogalmaznak [99]: minden sebezhető kriptovaluta-közösségnek haladéktalanul meg kell kezdenie a posztkvantum kriptográfiára való átállást.

12 Mi az a pénz?

Mielőtt a kriptovaluták világába merülnénk, érdemes egy pillanatra megállni és feltenni azt a kérdést, hogy tulajdonképpen mi is az a pénz. A pénz fogalmának megértéséhez egy rendkívül szemléletes történelmi példát vehetünk alapul: a kínai papírpénz történetét, amelyet Marco Polo is megörökített útleírásában.

Marco Polo 1271 és 1295 között járt Kínában, és az 1298-ban íródott *Il Milione* (Cso-dák Könyve) című úti beszámolójában részletesen leírta a papírpénz működését – egy olyan rendszert, amely az akkori európaiak számára teljesen ismeretlen volt. A papírpénz ötlete azonban nem volt új: a Song-dinasztia idején, az 1020-as években már léteztek papírpénz-formák (az ún. *jiaozi*, amelyet szecsuanai kereskedők bocsátottak ki). Hsziào Cung (Xiaozong, 1162–1189) császár uralkodása alatt, az 1160-as években a rendszert tovább fejlesztették és központilag szabályozták. Marco Polo leírásából kiderül, hogy a pénz kibocsátásának folyamata szigorúan szabályozott volt. Minden egyes papírdarabra számos hivatalnoknak – akiknek ez volt a kötelessége – fel kellett írnia a nevét és rá kellett tennie a pecsétjét. Végül a Nagy Khán által megbízott főtiszt smaragdvrörös festékekkel kent be egy pecsétet, és rányomta a papírra. Ettől a pillanattól kezdve a pénz hitelesnek számított, a hamisítást pedig halálbüntetéssel szankcionálták.

Ez a rendszer számos olyan tulajdonsággal rendelkezett, amelyek a modern pénzrendszerek – és később a kriptovaluták – szempontjából is relevánsak. A Nagy Khán minden évben hatalmas mennyiséget készíttetett ebből a pénzből, ami lényegében semmibe sem került neki. A papírpénzt általánosan elfogadtatta az összes királyságában, tartományában és területén, bárhol, ahol hatalma kiterjedt. Senki nem mert megtagadni az elfogadásukat, és mindenki könnyedén használta őket, hiszen a birodalom egész területén forgalomban voltak. Árut eladni és vásárolni velük éppolyan könnyű volt, mintha tiszta aranyérmék lettek volna.

A kereskedelem szempontjából a rendszer úgy működött, hogy a más országból érkező kereskedők – akik arannyal, ezüsttel, drágakövekkel és gyöngyökkel érkeztek – kizárólag az uralkodónak adhatták el portékáikat. A Khán tizenkét szakértőt választott ki erre a feladatra, akik felbecsülték az árucikkeket, majd az uralkodó bőséges árat fizetett értük papírpénzben. A kereskedők elfogadták az árat, mert senkitől sem kaptak volna jobbat, a kifizetés pedig azonnal megtörtént. A papírpénzzel azután bármit megvehettek az egész birodalmon belül, miközben sokkal könnyebb is volt cipelni az útjuk során.

Így lett végtelen kincse a Khánnak, miközben az általa kifizetett pénz előállítására semmibe sem került neki. Többször is kihirdették évente a városban, hogy bárki, akinek aranya, ezüstje, drágakövei vagy gyöngyei vannak, elviheti azokat a kincstárba, és szép árat kap értük. A tulajdonosok örömmel tették ezt, mert nem találtak volna más vásárlót, aki ennyit fizetett volna – így szinte az összes értéktárgy a Nagy Khán birtokába került. Ha bármelyik papírdarab megrongálódott, a tulajdonos elvitte a pénzverdébe, és az érték három százalékának megfelelő díjat fizetve új darabokat kapott helyettük. Ha pedig bárkinek szüksége volt aranyra, ezüstre, drágakövekre vagy gyöngyökre – például étkészletek vagy övek készítéséhez –, elment a pénzverdéhez, és annyit vásárolt amennyit akart, papírpénzzel fizetve.

Ebben a történelmi példában már felismerhetjük a modern pénz alapvető jellemzőit: a pénz értéke nem az anyagából fakad, hanem a mögötte álló hatalom és bizalom adja. A kibocsátó hatóság hitelesíti (pecsét, aláírás), a hamisítást szigorúan büntetik, és az elfogadás kényszere biztosítja a forgalmat. Ahogyan majd látni fogjuk, a kriptovaluták esetében ezeket a funkciókat – hitelesítés, hamisítás elleni védelem, általános elfogadottság

– matematikai és kriptográfiai eszközökkel valósítják meg, központi hatóság helyett egy elosztott hálózat konszenzusára támaszkodva.

13 Megbízható számítások

A blokkláncok megértéséhez elengedhetetlen megismernünk azokat az alapvető informatikai kérdéseket, amelyek az elosztott rendszerek megbízhatóságával foglalkoznak. Ezek a problémák jóval a kriptovaluták megszületése előtt felmerültek, és a megoldásukra kidolgozott elméletek képezik a blokkláncok konszenzus-mechanizmusainak alapját.

13.1 Neumann János és a megbízhatatlan alkatrészekből épített megbízható rendszerek

Neumann János (1903–1957) egyik utolsó tanulmánya a *Probabilistic Logics and the Synthesis of Reliable Organisms from Unreliable Components* [41] címet viselte. Ebben a munkájában arra a kérdésre kereste a választ, hogy miként építhetők fel megbízhatatlan alkatrészekből megbízható működésű gépek. A probléma nagyon is gyakorlati volt: az elektronikus számológépek aktív elemeinek tízezrei másodpercenként sok százezer műveletet végeztek, és bár egy-egy alkatrész meghibásodásának valószínűsége kicsi volt, technikailag teljesen elkerülhetetlen volt, hogy időnként hibák lépjenek fel.

Neumann János megmutatta, hogy az efajta hibák ellen védelmet nyújthat a redundancia elve: ha egy művelet párhuzamosan több alkatrész-sorozat végéig el, a művelet végrehajtása után egymás között összehasonlítják a kapott eredményeiket. Ha meghibásodás folytán nem minden vonalon egyeznek meg az eredmények, a rendszer statisztikusan döntést hoz: a legtöbbször kapott eredményt fogadja el helyesnek, és ezzel számol minden vonalon tovább. A műszaki gyakorlatban is alkalmazzák ezt az elvet: bizonyos szerkezeti elemeket megdupláznak (vagy megtöbbszöröznék), és mivel annak a valószínűsége, hogy mindegyik szerkezeti elem egyidejűleg hibásodik meg, elegendően kicsi, a rendszer egészének hibázási valószínűsége tetszőlegesen csökkenthető.

Ebben a megközelítésben jelenik meg először a többségi szavazás fogalma, ami – mint majd látni fogjuk – a blokkláncok konszenzus-mechanizmusainak központi gondolata. A multiplexelés (több párhuzamos feldolgozó egység alkalmazása) és a többségi döntés gondolata nemcsak a gépi automaták, hanem az emberi agy működésében is megjelenik: az agy is redundáns neuronhálózatokra épül, amelyek többségi döntéssel kompenzálják az egyes neuronok megbízhatatlanságát.

13.2 Időmérés elosztott rendszerekben

Az elosztott rendszerek következő alapvető kihívása az események sorrendjének meghatározása. Leslie Lamport 1978-as tanulmánya (*Time, Clocks, and the Ordering of Events in Distributed Systems*) [42] az egyik legmeghatározóbb munka ezen a területen. A központi kérdés a következő: ha több, egymástól független számítógép működik együtt egy hálózatban, hogyan állapítható meg a konkurens események sorrendje? Hogyan kell erőforrásokat lefoglalni és felszabadítani úgy, hogy a rendszer konzisztens maradjon?

Lamport felismerése az volt, hogy az idő fogalma – amelyet a hétköznapi gondolkodásban magától értetődőnek tartunk – elosztott rendszerekben nem triviális. Fizikailag szétválasztott számítógépek óráit nem lehet tökéletesen szinkronizálni, ezért az események sorrendjét nem alapozhatjuk fizikai időre. Ehelyett Lamport bevezette a „happened

before” (megelőzi) relációt, amelyet a $\rightarrow$ szimbólummal jelölt. A reláció definíciója három szabályra épül: (1) ha a és b ugyanazon folyamat eseményei, és a megelőzi b -t a folyamatban, akkor $a \rightarrow b$; (2) ha a egy üzenet elküldése és b ugyanazon üzenet fogadása egy másik folyamatban, akkor $a \rightarrow b$; (3) ha $a \rightarrow b$ és $b \rightarrow c$, akkor $a \rightarrow c$ (transzitivitás). Két esemény konkurens, ha egyik sem előzi meg a másikat. A $\rightarrow$ reláció tehát az események halmazán egy irreflexív parciális rendezést definiál – nem minden eseménypár hasonlítható össze, és ez az elosztott rendszerek alapvető sajátossága.

A parciális rendezés önmagában nem elegendő sok gyakorlati problémához, ezért Lamport bevezette a logikai órákat: minden P_i folyamathoz tartozik egy C_i számláló, és két egyszerű szabályt kell betartani. (IR1) Minden folyamat a saját egymást követő eseményei között növeli az óráját. (IR2) Ha egy folyamat üzenetet küld, az üzenethez időbélyeget csatol (az aktuális óraértéket); az üzenetet fogadó folyamat az óráját az aktuális értéke és az üzenet időbélyege közül a nagyobbra (plusz egy) állítja. Ezekkel a szabályokkal teljesül az ún. Clock Condition: ha $a \rightarrow b$, akkor $C(a) < C(b)$. A logikai órák segítségével a parciális rendezés kiegészíthető teljes rendezéssé – holtverseny esetén a folyamatok egy tetszőleges, de rögzített sorrendje dönt.

Lamport megmutatta, hogy a teljes rendezés segítségével megoldható például a kölcsönös kizárás problémája elosztott környezetben, központi koordinátor nélkül: a folyamatok időbélyeges kéréseket küldenek egymásnak, és az erőforrást mindig az kapja meg, akinek a kérése a legkisebb időbélyeget viseli – feltéve, hogy minden más folyamattól kapott már későbbi időbélyegű üzenetet. Ez a megoldás közvetlenül inspirálta a blokkláncoknál alkalmazott tranzakció-sorrendezési mechanizmusokat.

Szintén 1978-ban jelent meg Lamport másik meghatározó munkája (*The Implementation of Reliable Distributed Multiprocess Systems*) [43], amelyben egy konkrét algoritmust adott arra, hogyan implementálható bármilyen rendszer úgy, hogy az egyedi folyamatok vagy kommunikációs vonalak meghibásodása ellenére is helyesen működjön tovább. A tanulmány két fő célt különböztet meg: (1) a rendszer akkor is működjön, ha egy vagy több komponens teljesen leáll („failure”); (2) a rendszer akkor is helyesen működjön, ha egy vagy több komponens hibásan működik, azaz rossz eredményt ad („malfunction”).

Lamport megoldása a felhasználói gép (*user machine*) fogalmára épül: a teljes rendszer viselkedését egy szekvenciális, determinisztikus állapotgéppel definiáljuk, amely parancsokat fogad és válaszokat generál. Ezt az állapotgépet több, fizikailag elkülönített folyamat – az ún. bookkeeper-ek – párhuzamosan szimulálja. Minden bookkeeper a saját másolatát tartja fenn a felhasználói gép állapotának, és az időbélyeges parancsokat azonos sorrendben hajtja végre. Így a rendszer logikailag egyetlen szekvenciális géppel ekvivalens, fizikailag viszont elosztott és hibatűrő.

A megbízhatóság kulcsa a többségi részgráf (*majority graph*) fogalma: a rendszer mindaddig helyesen működik, ameddig a bookkeeper-ek és a köztük lévő kommunikációs vonalak egy többségi részhalmaza megfelelően funkcionál. Egy egyszerű példával élve: ha három gépünk van és kettő szavazatát kell beszereznünk egy művelet jóváhagyásához (például egy repülőjegy lefoglalásához), akkor a rendszer még egyetlen gép kiesése esetén is működőképes marad. Formálisan Lamport az ún. RC feltételt (Reliability Condition) fogalmazta meg: ha létezik egy többségi részgráf, amely megfelelően működik egy adott időintervallumban, akkor minden parancs, amelyet a részgráf bármelyik bookkeeper-je megkap, el lesz fogadva és végre lesz hajtva az összes többi bookkeeper által is. A kulcsfeltétel tehát: mindig létezzen többségi részgráf a hálózatban – ekkor a csomópontok nem fognak összezavarodni.

Egy lényeges tulajdonsága az algoritmusnak, hogy egy bookkeeper soha nem kérde-

zi meg, vajon egy másik komponens működik-e – egyszerűen követi a szabályokat, és a rendszer struktúrája garantálja a helyességet. A tranziens hibákat (pl. elveszett üzeneteket) nem kell detektálni; az algoritmus természeténél fogva tolerálja őket. Ez a gondolat közvetlenül megjelenik a blokkláncok tervezésében is: a csomópontok nem ellenőrzik egymás állapotát, hanem a protokoll szabályai és a többségi szavazás garantálja a rendszer konzisztenciáját.

A modell egyszerű feltételezésekre épül: ha elküldtek egy üzenetet, az be is érkezik (a kommunikációs csatorna megbízható), és van időbeli sorrendiség az események között. Erre az alapra építhető az állapotgép-replikáció (*state machine replication*) módszere: minden csomópont ugyanazt az állapotgépet hajtja végre, és a konszenzus biztosítja, hogy mindenki azonos állapotba kerüljön. Ez az architektúra közvetlen előzménye a blokklánc-technológiának.

13.3 A bizánci tábornokok problémája

1982-ben Leslie Lamport, Robert Shostak és Marshall Pease publikálta a *The Byzantine Generals Problem* [44] című tanulmányt, amely az elosztott rendszerek hibatűrésének egyik legfontosabb elméleti alapja lett. A probléma egy hadászati allegóriával fogalmazható meg: a bizánci hadsereg több divíziója táborozik egy ellenséges város körül, minden divíziót saját tábornoka irányít. A tábornokok kizárólag hírvivőkkel tudnak egymással kommunikálni. Az ellenség megfigyelése után közös haditervet kell alkotniuk – de néhányuk áruló lehet, aki megpróbálja megakadályozni, hogy a lojális tábornokok megegyezésre jussanak.

A probléma formális megfogalmazása két feltételt tartalmaz (az ún. interaktív konzisztencia feltételek):

- IC1: Minden lojális tábornok ugyanazt a tervet hajtja végre.
- IC2: Ha a főparancsnok lojális, akkor minden lojális tábornok az ő parancsát hajtja végre.

A tanulmány legmeglepőbb eredménye, hogy szóbeli üzenetekkel (oral messages) – ahol az üzenet tartalma teljes mértékben a küldő kontrollja alatt áll, tehát egy áruló bármit küldhet – a probléma csak akkor oldható meg, ha a résztvevők több mint kétharmada lojális. A szóbeli üzenetek három feltételezésre épülnek: (A1) minden elküldött üzenet helyesen megérkezik; (A2) a fogadó tudja, ki küldte az üzenetet; (A3) az üzenet hiányát észlelni lehet. Lamporték bebizonyították, hogy három tábornok esetén, ha egy áruló van közöttük, a probléma megoldhatatlan. Az érvelés elegánsan egyszerű: ha a főparancsnok „támadj” parancsot küld az 1. hadnagynak, de a 2. hadnagy (áruló) azt állítja, hogy ő „visszavonulj” parancsot kapott, az 1. hadnagy nem tudja megkülönböztetni ezt a helyzetet attól, amelyben a főparancsnok az áruló és különböző parancsokat küldött a két hadnagynak. Általánosan: ha m áruló van a rendszerben, az ún. OM(m) algoritmus (Oral Messages) csak akkor garantálja a helyes konszenzust, ha összesen több mint $3m$ tábornok van, vagyis legalább $3m + 1$ résztvevő szükséges.

A helyzet alapvetően megváltozik, ha aláírt üzeneteket (signed messages) használunk. Ekkor egy negyedik feltételezést is bevezetünk: (A4) egy lojális tábornok aláírása nem hamisítható, és az aláírt üzenet bármilyen módosítása kimutatható. Aláírásokkal a korábbi lehetetlenségi eredmény nem érvényes: három tábornokkal és egy árulóval is létezik megoldás. Az SM(m) algoritmus (Signed Messages) tetszőleges számú áruló esetén működik, amennyiben legalább $m + 2$ tábornok van összesen. Az algoritmus lényege: a főparancsnok

aláírva elküldi a parancsát, majd minden hadnagy hozzáfűzi a saját aláírását és továbbítja a többieknek. Ha valaki két különböző, a főparancsnok által aláírt parancsot kap, azonnal tudja, hogy a főparancsnok áruló – és ilyenkor a megegyezés szerinti alapértelmezett parancsot (pl. „visszavonulás”) hajtja végre.

A többség definiálása tehát kulcskérdés: minden csomópont a kapott üzenetekből számítja ki a saját valóságát, és a többségi szavazat alapján hoz döntést. A jóhiszemű résztvevők akkor jutnak konszenzusra, ha elegendő számban vannak. A digitális aláírások alkalmazása radikálisan javítja a helyzetet, mert az árulók nem tudnak mások nevében hamis üzeneteket küldeni, így a szükséges lojális résztvevők aránya jóval alacsonyabb lehet.

Ez az elméleti keret – a redundancia, a többségi szavazás, a bizánci hibatűrés és a digitális aláírások kombinációja – alkotja a blokkláncok konszenzus-mechanizmusainak alapját, amelyekről a következő fejezetekben részletesen szó lesz.

14 A digitális aláírás és a blokkláncok

2008. október 31-én egy Satoshi Nakamoto álnevet használó személy (vagy csoport) publikálta a Bitcoin whitepaper-t [45], amely egy teljesen új megközelítést javasolt az elektronikus pénz megvalósítására. A tanulmány központi gondolata rendkívül elegáns: *„We define an electronic coin as a chain of digital signatures.”* – vagyis egy elektronikus érmét nem más definiál, mint digitális aláírások láncolata. A digitális aláírás hamisíthatatlansága garantálja, hogy az elektronikus pénz nem másolható és nem hamisítható.

A rendszer működése a következő: minden tulajdonos úgy adja tovább az érmét a következő személynek, hogy digitálisan aláírja az előző tranzakció hash-ét és a következő tulajdonos nyilvános kulcsát, majd ezeket hozzáfűzi az érme tranzakciós láncához. Ahogyan a whitepaper fogalmaz: *„Each owner transfers the coin to the next by digitally signing a hash of the previous transaction and the public key of the next owner and adding these to the end of the coin.”* A kedvezményezett (payee) az aláírások ellenőrzésével egyértelműen vissza tudja követni a tulajdonjog teljes láncát.

14.1 A dupla-költés problémája

Eddig a konstrukció elegáns, de van egy alapvető probléma, amelyet meg kell oldani: a dupla-költés (double-spending). Fizikai érméknél ez a probléma nem létezik – ha odaadom valakinek a fémpénzemet, akkor az nálam már nincs meg. Digitális adatoknál viszont bármi másolható, tehát semmi sem akadályozza meg, hogy valaki ugyanazt a digitális érmét egyszerre két különböző személynek is elküldje.

A hagyományos megoldás egy megbízható központi hatóság („mint”) bevezetése, amely minden tranzakciót ellenőriz. Ahogy a whitepaper leírja: *„A common solution is to introduce a trusted central authority, or mint, that checks every transaction for double spending.”* Minden tranzakció után az érmét vissza kell küldeni a központi hatósághoz, amely új érmét bocsát ki, és csak a közvetlenül a hatóság által kibocsátott érmék tekinthetők érvényesnek.

A probléma ezzel a megoldással az, hogy az egész pénzrendszer sorsa egyetlen szervezettől függ, és minden tranzakciónak rajtuk kell átmennie – pontosan úgy, mint egy banknál. Ahogyan azt a Marco Polo-féle példában láttuk, a központi hatóságra épülő pénzrendszer sebezhetővé teszi a felhasználókat: ha a hatóság visszaél a hatalmával, manipulálja a rendszert, vagy egyszerűen megszűnik működni, az egész rendszer összeomlik.

14.2 Az elosztott megoldás

Satoshi Nakamoto megoldása az volt, hogy a központi hatóság helyett egy elosztott főkönyvet (distributed ledger) alkalmazott: ha a hálózat csomópontjainak többsége azt mondja, hogy egy tranzakció érvényes és beérkezett, akkor elfogadjuk érvényesnek. Ez visszavezet minket a bizánci tábornokok problémájához és a többségi szavazás elvéhez.

De hogyan biztosítjuk, hogy a szavazás tisztességes legyen, és ne lehessen olcsón manipulálni? A megoldás a proof-of-work (munkabizonyíték), amelynek ötlete eredetileg Adam Back nevéhez fűződik, aki 1997-ben javasolta a Hashcash [77] rendszert (formálisan 2002-ben publikálva) azzal a céllal, hogy megakadályozza az e-mail spamelést: ha minden e-mail küldéséhez számítási munkát kell végezni, a tömeges spam gazdaságilag értelmetlenné válik.

A Bitcoin ugyanezt az elvet alkalmazza a konszenzus eléréséhez. Ahogyan a whitepaper fogalmaz: „*Proof-of-work is essentially one-CPU-one-vote.*” – vagyis a szavazati jog nem személyekhez, hanem számítási kapacitáshoz kötött. Ez megakadályozza, hogy valaki olcsón, sok hamis identitás létrehozásával (ún. Sybil-támadás) manipulálja a szavazást. A leghosszabb lánc mindig az, amelyikbe a legtöbb proof-of-work munka lett befektetve (*longest chain = greatest proof-of-work*), és a hálózat mindig ezt fogadja el az érvényes történetként.

Ebből következik a Bitcoin biztonságának alapvető feltétele: amíg a jóhiszemű csomópontok rendelkeznek a számítási kapacitás többségével, addig mindig övék lesz a leghosszabb lánc, és a támadók nem tudják felülrni a tranzakciós történetet. Ez a gondolat közvetlenül visszavezethető Neumann és Lamport munkáira: a megbízhatatlan alkatrészekből megbízható rendszert a redundancia és a többségi szavazás teszi lehetővé.

15 Konszenzus mechanizmusok

Az előző fejezetekben megismertük az elosztott rendszerek elméleti alapjait: Neumann redundancia-elvét, Lamport logikai óráit és állapotgép-replikációját, valamint a bizánci tábornokok problémáját. Ebben a fejezetben azt vizsgáljuk, hogy ezek az elméleti eredmények hogyan valósulnak meg a blokklánc-technológiában. A fejezet a következő alapvető kérdésekre keresi a választ:

- Mi az a blokklánc, és hogyan működik?
- Mekkora késleltetést tolerálhat a rendszer?
- Hogyan definiálják a többséget?
- Hány csaló (bizánci hibás csomópont) ellen lehet védekezni?
- Hogyan fogadják el az állapotváltozásokat?
- Milyen korlátokat adhatunk a biztonságos működéshez?
- Mi történik, ha alternatív valóságok (elágazások) jelennek meg?
- Milyen centralizáló erők léphetnek fel?
- Hogyan lehet a támadók ellen védekezni, és milyen pénzügyi ösztönzőkkel?

15.1 A blokklánc alapstruktúrája

Tegyük fel, hogy hálózatba kötött számítógépeink vannak, amelyek egy peer-to-peer (P2P) hálózatot alkotnak. Egyetlen központi szerver sincs – minden csomópont egyenrangú. Amikor valaki tranzakciót szeretne küldeni (például kriptovalutát utalni), a folyamat a következő lépésekből áll:

1. A tranzakció blokk formájában jelenik meg (egy blokk több tranzakciót is tartalmazhat).
2. A blokk bekerül a P2P hálózatba és elterjed a csomópontok között.
3. A csomópontok leellenőrzik a blokk érvényességét (érvényes aláírások, nincs duplaköltés, stb.).
4. Ha a blokk érvényes és a konszenzus-mechanizmus szabályai teljesülnek, a blokkot hozzáadják a lánchoz.

Minden blokk a következő három alapvető elemet tartalmazza:

- Az előző blokk hash-e – ez köti össze a blokkokat egy láncba, és garantálja, hogy a korábbi blokkok tartalmát nem lehet megváltoztatni anélkül, hogy az összes következő blokk hash-e is megváltozna.
- N darab tranzakció – a blokk hasznos tartalma.
- Valamilyen bizonyíték (proof) – amely a konszenzus-mechanizmustól függ, és amelynek előállítása garantálja, hogy a blokkot a szabályok szerint hozták létre.

A blokkok láncolása biztosítja a rendszer integritását: ha valaki megpróbálna módosítani egy korábbi tranzakciót, az megváltoztatná annak a bloknak a hash-ét, ami érvénytelenítené az összes utána következő blokkot. Ezért egy támadónak az összes rákövetkező blokkot újra kellene számolnia – ami, mint látni fogjuk, a konszenzus-mechanizmus miatt gazdaságilag vagy számítástechnikailag megvalósíthatatlan.

15.2 Proof-of-work (munkabizonyíték)

A proof-of-work (PoW) az első és legismertebb konszenzus-mechanizmus, amelyet a Bitcoin alkalmaz. A lényege a következő: a bányász csomópontok együtt próbálják megtalálni az SHA-256 hash függvény egy speciális bemenetét, amely egy adott feltételt kielégít. Formálisan: keresünk egy nonce értéket, amelyre

$$H(\text{blokk-fejléc}||\text{nonce}) < \text{target}$$

ahol H az SHA-256 hash függvény, a target pedig egy előre meghatározott küszöbérték. Mivel a hash függvény kimenetét nem lehet előre megjósolni (a hash függvény egyirányú és pszeudóvéletlen kimenetet ad), az egyetlen módszer a nonce szisztematikus próbálgatása (brute force). A megoldás tehát a nonce érték, amellyel ha kiszámoljuk a blokk hash-ét, az kielégíti a nehézségi feltételt.

A nehézség (difficulty) beállítása kulcsfontosságú a rendszer stabilitásához. Minél nagyobb a nehézség (azaz minél kisebb a target), annál több próbálkozásra van szükség a megfelelő nonce megtalálásához, tehát annál lassabban születik új blokk. A Bitcoin esetében a nehézséget minden 2016 blokkonként (körülbelül két hetente) automatikusan úgy

állítják be, hogy a blokkidő átlagosan 10 perc maradjon. Ha a bányászok száma (és ezzel a hálózat összesített számítási kapacitása, az ún. hashrate) megnő, a nehézséget emelik; ha csökken, csökkentik. Ez biztosítja, hogy a blokkok keletkezésének üteme nagyjából konstans maradjon, függetlenül attól, hány bányász dolgozik éppen a hálózatban.

Amikor egy bányász megtalálja a megfelelő nonce-t – vagyis a hash kielégíti a feltételt –, az új blokk készen áll: elküldi a hálózatnak, a többi csomópont ellenőrzi (ami triviálisan gyors, hiszen egyetlen hash-számítás), és ha érvényes, hozzáfűzi a lánchoz. A bányász jutalmul kap néhány frissen kibocsátott kriptovalutát (block reward) és az adott blokkba foglalt tranzakciók díjait (transaction fees). Ez az a pénzügyi ösztönző, amely motiválja a bányászokat az energiaigényes számítási munka elvégzésére.

A proof-of-work biztonsága azon alapul, hogy a számítási munka költséges, de az ellenőrzés olcsó. Egy támadónak, aki felül akarja írni a lánc történetét, az összes becsületes bányász együttes számítási kapacitásánál többel kellene rendelkeznie – ez az 51%-os támadás feltétele. Amíg a becsületes csomópontok rendelkeznek a hashrate többségével, a leghosszabb (legtöbb munkát tartalmazó) lánc mindig az ő kezükben lesz, és a támadók alternatív valósága nem tud felülkerekedni.

15.3 Proof-of-stake

A proof-of-stake (PoS) a proof-of-work alternatívája, amely a számítási munka helyett gazdasági letételre alapozza a konszenzust. A működési elv a következő: a résztvevők (validátorok) kriptovalutát fagyasztanak be („stake-elnek”) a láncon, ezzel jelezve, hogy érdeklükben áll a rendszer helyes működése. A validátorokat a befagyasztott összeg arányában választják ki blokkok javaslására és szavazásra – minél nagyobb valakinek a stake-je, annál nagyobb valószínűséggel kerül sorra.

A biztonság logikája analóg a PoW-val, de a „munka” helyett a „tőke” a szűk erőforrás: egy támadónak a teljes befagyasztott mennyiség 51%-át kellene megszereznie ahhoz, hogy a hálózatot manipulálhassa. Mivel a stake-elt valuta értéke a lánc biztonságától függ, egy sikeres támadás éppen azt az eszközt értéktelensévé, amelyet a támadó felhalmozott – ez erős gazdasági elrettentő hatást jelent. Emellett a legtöbb PoS rendszerben az a validátor, aki szabályellenesen viselkedik (pl. két különböző blokkra szavaz egyszerre), büntetésként elveszíti a befagyasztott tőkéjének egy részét vagy egészét (ezt hívják „slashing”-nek).

A blokkgyártásért és a szavazásért jutalom jár: a validátorok tranzakciós díjakat és/vagy újonnan kibocsátott kriptovalutát kapnak, ami ösztönzi a részvételt. A PoS előnye a PoW-val szemben, hogy nem igényel hatalmas energiafogyasztást, hiszen nincs szükség tömeges hash-számításra. Hátránya, hogy a „gazdagok gazdagabbak lesznek” problémát veti fel (minél több stake-ed van, annál több jutalmat kapsz), és a „nothing-at-stake” probléma is felmerül: egy validátornak elvileg nem kerül semmibe egyszerre több elágazásra szavazni, szemben a PoW-val, ahol a bányász számítási kapacitása véges. Ezt a modern PoS rendszerek (pl. Ethereum) a slashing mechanizmussal kezelik.

15.4 Egyéb konszenzus mechanizmusok

A proof-of-work és a proof-of-stake mellett számos más konszenzus-mechanizmus is létezik, amelyek különböző kompromisszumokat kínálnak a decentralizáció, a hatékonyság és a biztonság között.

A delegált proof-of-stake (DPoS) esetében a token-tulajdonosok nem közvetlenül vesznek részt a blokkgyártásban, hanem szavazataikkal validátorokat választanak, akik a ne-

vükben járnak el. Ez növeli a hatékonyságot (kevesebb validátor, gyorsabb konszenzus), de csökkenti a decentralizációt. A DPoS-t alkalmazza például az EOS [78] (21 választott blokk-producer) és a Tron [79] (27 szuper-képviselő). Más megközelítések közé tartozik a Proof of Burn (kriptovaluta megsemmisítése a részvételi jog megszerzéséért, ilyen például a Counterparty [80]), a Proof of Activity (PoW és PoS hibridje, a Decred [81] alkalmazza) és a Proof of Luck (véletlenszám-generátorokra épülő kiválasztás). A Stellar Consensus Protocol [82] a föderatív bizánci megegyezés (Federated Byzantine Agreement) elvén működik, ahol a csomópontok maguk választják meg, kiben bíznak.

15.4.1 Practical Byzantine Fault Tolerance (PBFT)

A PBFT algoritmus [83] közvetlenül a bizánci tábornokok problémáját oldja meg egy gyakorlati környezetben. Miguel Castro és Barbara Liskov 1999-es tanulmánya azért volt áttörés, mert korábban a bizánci hibatűrő protokollokat túl lassúnak tartották a valós alkalmazásokhoz. Castro és Liskov megmutatták, hogy egy hatékony, háromfázisú protokollal a probléma gyakorlatilag is megoldható.

Az algoritmus három fázisból áll: (1) pre-prepare – a kijelölt vezető (primary) sorrendszámot rendel a kliens kéréséhez és elküldi a többi replikának; (2) prepare – a replikák megerősítik, hogy látták a kérést az adott sorrendszámmal; (3) commit – ha egy replika elegendő prepare üzenetet kapott ($2f$ darabot, ahol f a maximálisan tolerált hibás csomópontok száma), véglegesíti a műveletet. A protokoll összesen $3f + 1$ replikát igényel f bizánci hiba tolerálásához.

Két alapvető tulajdonságot garantál: safety (a jóhiszemű csomópontok soha nem fogadnak el ellentmondó állapotot – ez akkor is teljesül, ha a hálózat tetszőlegesen lassú) és liveness (a rendszer végül mindig előrehalad, feltéve hogy a hálózat részlegesen szinkron). Ha a primary hibásan viselkedik, a protokoll view change mechanizmussal új vezetőt választ.

Castro és Liskov a BFT-NFS rendszerrel demonstrálták a protokoll gyakorlati használatosságát: egy bizánci hibatűrő NFS (hálózati fájlrendszer) implementáció teljesítménye mindössze 3%-kal volt lassabb, mint a nem replikált változat. Ez megcáfolta a korábbi feltételezést, miszerint a bizánci hibatűrés túl drága lenne a gyakorlatban. A PBFT állapotgép-replikálással (state machine replication) éri el a konszenzust: minden csomópont ugyanazt az állapotgépet futtatja, és a protokoll biztosítja, hogy mindenki azonos sorrendben hajtsa végre a tranzakciókat. A PBFT lett az alapja számos későbbi blokklánc-protokollnak, köztük a Hyperledger Fabric korai verzióinak és a Tendermint-nek.

15.4.2 Tendermint és az FLP lehetetlenség

Hogyan lehet bányászat nélkül konszenzusra jutni? A Tendermint protokoll [84] a bizánci tábornokok problémájának egy modern, gyakorlati megoldása. Validátorokat definiál, akik kriptovalutát allokálnak (stake-elnek) a szavazás előtt, és mindenki annyit szavazhat, amennyi kriptovalutát befagyasztott.

A Tendermint tervezésekor fontos elméleti korlátot kellett figyelembe venni: az FLP lehetetlenségi tételt [85] (Fischer, Lynch, Paterson, 1985), amely kimondja, hogy teljesen aszinkron rendszerben – ahol az üzenetek késleltetésére semmilyen felső korlát nincs – lehetetlen determinisztikus konszenzust elérni, ha akár egyetlen hibás csomópont is van.

Az FLP tétel modellje a következő: N folyamat aszinkron üzenetküldéssel kommunikál, minden folyamatnak van egy bemeneti értéke (0 vagy 1), és a protokollnak biztosítania kell, hogy a nem hibás folyamatok megegyezzenek egyetlen közös döntési értékben.